



INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Licenciatura em Engenharia Biomédica

Sistemas de Informação e Base de Dados Aplicados à Saúde

Ano Letivo 2025/2026

GIRA – Sistema Web de Apoio ao Inventário Hospitalar de Equipamentos Médicos

Professores:

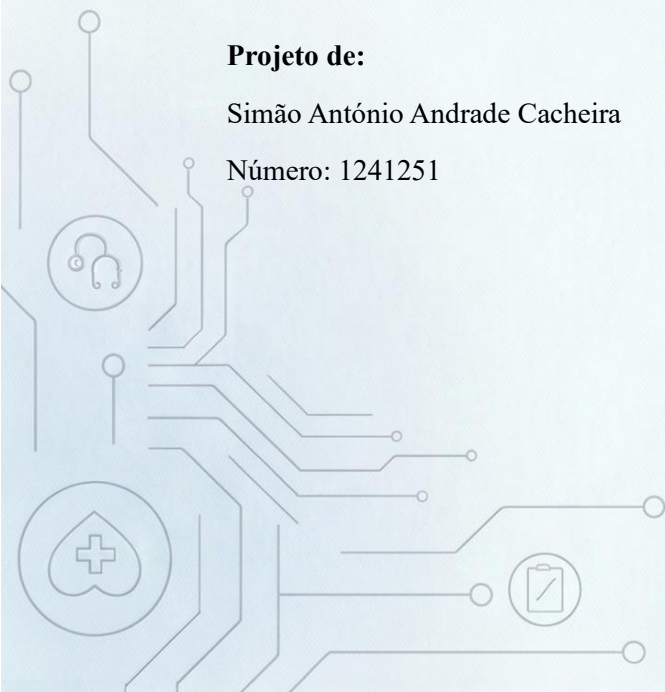
Pedro Guimarães

Nuno Morgado

Projeto de:

Simão António Andrade Cacheira

Número: 1241251



isep

Instituto Superior de
Engenharia do Porto

Índice

Introdução e Enquadramento.....	4
Contexto e Justificação.....	4
Objetivos do Projeto.....	4
Âmbito do Sistema.....	4
Arquitetura e Estrutura Tecnológica.....	6
Arquitetura de Software.....	6
Camada de Apresentação (Interface/Frontend).....	6
Camada de Lógica de Negócio (Backend em PHP).....	6
Camada de Persistência de Dados (Base de Dados MySQL).....	6
Organização de Diretórios.....	6
/public/ (Área Pública e Autenticação).....	8
/private/ (Área Reservada e Módulos).....	8
/assets/ (Recursos Estáticos e Identidade Visual).....	8
Front Office (Área Pública).....	9
Conceito do Website Institucional.....	9
Mecanismo de Atualização Dinâmica.....	9
Modelação e Engenharia da Base de Dados (MySQL).....	10
Modelo Relacional e Esquema DBML.....	10
Dicionário de Dados e Regras de Normalização.....	14
Restrições de Integridade Aplicadas no Modelo.....	15
Restrições de Entidade (UNIQUE).....	15
Restrições de Integridade Referencial (Comportamento ON DELETE).....	15
Restrições Não Representáveis no Modelo Relacional.....	16
Restrições de Formato e Padrão (Expressões Regulares).....	16
Restrições Cronológicas e Temporais.....	16
Restrições de Domínio de Valores Lógicos.....	16
Consultas SQL Aplicadas.....	17
Consulta com Junção de Tabelas (JOIN).....	17
Consulta com Subconsulta (Subquery).....	17
Back Office (Área Privada e Módulos Funcionais).....	19
Autenticação e Segurança.....	19
Mecanismo de Login e <i>Hashing</i> de Palavras-passe.....	19
Gestão de Sessões e Proteção de Diretórios.....	19
Processo de Encerramento (Logout).....	19
Dashboard de Gestão (Painel Analítico).....	20

Conceção Técnica e Lógica de Negócio.....	20
Conceção Visual e Interface Gráfica	20
Módulos de Gestão Operacional (CRUDs)	20
Gestão de Equipamentos Médicos.....	21
Gestão de Localizações (Mapeamento Físico).....	21
Gestão de Fornecedores e Contratos.....	21
Gestão Documental e Uploads	22
Metodologia de Armazenamento (Sistema de Ficheiros vs. BLOB).....	22
Associação Relacional ao Parque Tecnológico	22
Gestão de Espaço e Remoção Física (unlink).....	22
Pesquisa Avançada e Filtragem Combinada	22
Arquitetura do Motor de Busca (DataTables)	23
Lógica de Filtragem Combinada e Cruzamento de Dados.....	23
Funcionalidades Opcionais e de Valorização.....	24
Exportação Analítica (Relatórios em Excel/CSV)	24
Motor de Auditoria (Logs) e Restauração de Dados (<i>Soft Deletes</i>).....	24
Armazém Técnico e Alertas de Rutura	24
Calendário Dinâmico de Manutenções.....	24
Segurança Transaccional (Propriedades ACID)	25
Interface Adaptativa (Modo Escuro / <i>Dark Mode</i>)	25
Controlo de Versões e Histórico de Desenvolvimento (Git)	26
Metodologia de Trabalho.....	26
Análise de Commits e Comprovação de Evolução	26
Conclusões e Trabalho Futuro	27
Considerações Finais.....	27
Limitações e Propostas de Melhoria.....	27
Referências.....	28

Índice de Figuras

Figura 1 - Diagrama Entidade-Associação com a notação Crow's Foot (Pé de Galinha) da Gira.	14
--	----

Introdução e Enquadramento

Contexto e Justificação

A gestão eficiente do parque tecnológico é um dos pilares fundamentais para a segurança e o bom funcionamento de qualquer unidade de saúde. Em contextos hospitalares de média dimensão, o departamento de Engenharia Clínica depara-se frequentemente com o desafio de supervisionar um inventário vasto e complexo, muitas vezes superior a 1500 equipamentos médicos, desde dispositivos de suporte de vida a ferramentas de diagnóstico.

Historicamente, e de forma ainda muito prevalente, o controlo destes ativos é realizado de forma descentralizada e fragmentada, recorrendo a múltiplas folhas de cálculo (Excel) dispersas por vários computadores e a registos em suporte de papel. Esta abordagem analógica acarreta riscos severos para a instituição, nomeadamente a elevada probabilidade de inconsistência de dados, a perda do rastreio de prazos de garantia e a dificuldade em garantir a conformidade durante auditorias de qualidade e segurança. A transição para uma plataforma digital centralizada surge, assim, como uma necessidade crítica. A digitalização elimina redundâncias, mitiga o erro humano e garante que a informação vital sobre o estado operacional, histórico e localização das máquinas está acessível em tempo real.

Objetivos do Projeto

O objetivo geral deste projeto consiste no desenvolvimento e implementação de uma aplicação *web* responsiva e centralizada, denominada Gira, utilizando a tríade tecnológica PHP, MySQL e o *framework* Bootstrap. O sistema visa modernizar o fluxo de trabalho associado à gestão de inventário e documentação técnica.

Para atingir este propósito, foram definidos os seguintes objetivos específicos:

- Garantir a Rastreabilidade: Mapear com precisão o ciclo de vida de cada equipamento, desde a sua aquisição e localização física no hospital, até ao seu eventual abate.
- Centralizar a Gestão Documental: Desenvolver um mecanismo seguro para arquivar e aceder instantaneamente a manuais técnicos, certificados de conformidade e guias de calibração.
- Otimizar o Controlo de Fornecedores e Garantias: Estruturar uma base de dados relacional que ligue cada ativo aos seus fabricantes e fornecedores, monitorizando ativamente os prazos de cobertura legal e contratos de assistência.
- Preparar Escalas de Evolução: Estabelecer uma arquitetura de dados robusta e normalizada que permita a futura integração de um módulo abrangente de Manutenção Computadorizada, focado na automatização de ordens de trabalho preventivas e corretivas.

Âmbito do Sistema

O abrangente do sistema Gira delimita-se à vertente de gestão operacional do parque tecnológico e à sua interligação com as infraestruturas, recursos humanos e parceiros da unidade de saúde. A plataforma revela o seu valor através de uma arquitetura modular, englobando os seguintes componentes funcionais:

- Módulo de Equipamentos: Registo detalhado e categorização de ativos por classe de risco, estado operacional, identificação técnica (códigos únicos, números de série e MAC addresses), etc.
- Módulo de Localizações: Mapeamento da estrutura física da unidade (blocos, pisos e salas), permitindo alocar e localizar rapidamente os dispositivos no terreno.
- Módulo de Fornecedores: Gestão de contactos comerciais, fabricantes e prestadores de assistência técnica especializada.
- Módulo de Documentação e Garantias: Repositório de ficheiros anexados diretamente à ficha de cada equipamento e acompanhamento de datas críticas com alertas visuais.
- Módulo de Armazém e Stock Técnico: Gestão de peças sobresselentes e consumíveis clínicos, mapeamento de compatibilidades com os equipamentos e monitorização de ruturas com base em limites mínimos de segurança.
- Módulo de Manutenção (Ordens de Trabalho): Registo do histórico de intervenções, acompanhamento de avarias (corretivas) e calendarização de revisões ou calibrações (preventivas).
- Módulo de Segurança e Acessos: Administração de contas de utilizador e definição de perfis de autorização (Administradores, Engenheiros/Técnicos e Médicos/Enfermeiros), com controlo rigoroso de acessos.
- Módulo de Auditoria e Relatórios: Registo cronológico imutável de todas as ações realizadas na plataforma (Logs) e capacidade de exportação de dados analíticos para formatos estruturados (CSV/Excel).
- Módulo de Backoffice de Conteúdos (CMS): Interface de gestão dinâmica que permite a atualização em tempo real dos textos e contactos do portal público institucional.
- Dashboard Analítico: Painel de controlo agregador que fornece aos gestores métricas imediatas sobre a distribuição, disponibilidade, taxa de operabilidade e estado global do inventário.
- Módulo de Localizações: Mapeamento da estrutura física da unidade (blocos, pisos e salas), permitindo alocar e localizar rapidamente os dispositivos no terreno.
- Módulo de Fornecedores: Gestão de contactos comerciais, fabricantes e prestadores de assistência técnica especializada.
- Módulo de Documentação e Garantias: Repositório de ficheiros anexados diretamente à ficha de cada equipamento e acompanhamento de datas críticas com alertas visuais.
- Dashboard Analítico: Painel de controlo agregador que fornece aos gestores métricas imediatas sobre a distribuição, disponibilidade e o estado global do inventário.

Arquitetura e Estrutura Tecnológica

Arquitetura de Software

O desenvolvimento do sistema Gira foi alicerçado numa arquitetura clássica e robusta de três camadas (*3-Tier Architecture*), promovendo uma separação rigorosa de responsabilidades. Esta abordagem de engenharia garante não só a segurança do sistema, mas também a sua escalabilidade para a futura integração de novos módulos clínicos.

Camada de Apresentação (Interface/Frontend)

Responsável pela interação direta com o utilizador, esta camada foi desenvolvida com HTML5 e CSS3, suportada pelo *framework* Bootstrap 5. Esta escolha assegurou a criação de uma interface limpa, intuitiva e totalmente responsiva, adaptável aos ecrãs de computadores e *tablets* utilizados no contexto hospitalar. A interatividade foi enriquecida com JavaScript e a biblioteca DataTables, permitindo a manipulação dinâmica do Document Object Model (DOM), a renderização de tabelas com paginação e pesquisa em tempo real, e a gestão fluida de modais interativos sem sobrecarregar o cliente.

Camada de Lógica de Negócio (Backend em PHP)

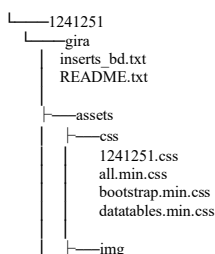
A inteligência central da aplicação reside no servidor, desenvolvida em PHP. O projeto adota uma segregação estrita entre as vistas de apresentação (ex: equipamentos.php ou armazenar.php) e os *scripts* de processamento (ex: processar_equipamento.php ou processar_encomenda.php). Estes controladores intermédios capturam os pedidos HTTP (POST/GET), aplicam regras rigorosas de validação de domínio (como formatos de MAC Address, verificações de NIFs e cálculos de *stock*), gerem o estado das sessões (`$_SESSION`) para controlo de permissões e devolvem o utilizador à interface correta através de redireccionamentos.

Camada de Persistência de Dados (Base de Dados MySQL)

O armazenamento persistente da informação é garantido por um motor de base de dados relacional (MySQL/MariaDB). A ponte de comunicação entre a lógica de negócio em PHP e a Base de Dados é estabelecida de forma centralizada e exclusiva através da extensão PDO (*PHP Data Objects*). Esta implementação é vital para a segurança da plataforma, uma vez que todas as interações e operações CRUD (Create, Read, Update, Delete) recorrem obrigatoriamente a *Prepared Statements* (declarações parametrizadas), eliminando por completo a vulnerabilidade a ataques de *SQL Injection*.

Organização de Diretórios

Para cumprir as melhores práticas de desenvolvimento *web* e refletir a complexidade do sistema construído, a árvore de ficheiros do projeto foi segmentada de forma lógica e segura. Esta hierarquia física espelha o controlo de acessos da própria plataforma.



- Dashboard.png
- favicon.png
- gatos_pingo_doce.jpg
- gato_pingo_doce.jpg
- Gemini_icon.png
- Hospital_building.jpg
- js
 - 1241251.js
 - bootstrap.bundle.min.js
 - datatables.min.js
 - jquery-3.7.1.min.js
 - search.htm
- webfonts
 - fa-brands-400.woff2
 - fa-regular-400.woff2
 - fa-solid-900.woff2
 - fa-v4compatibility.woff2
- private
 - dashboard.php
 - db.php
 - layout.php
 - logout.php
 - modals.php
- armazem
 - armazem.php
 - processar_artigo.php
 - processar_encomenda.php
- backoffice_publico
 - backoffice_publico.php
 - processar_conteudos.php
- documentos
 - documentos.php
 - eliminar_documento.php
 - processar_documento.php
- uploads
 - equipamentos
 - detalhes_equipamento.php
 - eliminar_equipamento.php
 - equipamentos.php
 - processar_edicao_equipamento.php
 - processar_equipamento.php
 - suspender_equipamento.php
- fornecedores
 - eliminar_fornecedor.php
 - fornecedores.php
 - processar_edicao_fornecedor.php
 - processar_fornecedor.php
- garantias
 - eliminar_garantia.php
 - garantias.php
 - processar_garantia.php
- historico
 - historico.php
 - processar_limpeza.php
 - restaurar_registro.php
- localizacoes
 - eliminar_localizacao.php
 - localizacoes.php
 - processar_edicao_localizacao.php
 - processar_localizacao.php
- manutencao
 - eliminar_ot.php
 - manutencao.php
 - processar_fecho_ot.php
 - processar_manutencao.php
- meu_perfil
 - meu_perfil.php
 - processar_meu_perfil.php
- perfis
 - eliminar_perfil.php
 - perfis.php
 - processar_edicao_perfil.php
 - processar_perfil.php
- relatorios
 - exportar_relatorio.php
 - relatorios.php
- utilizadores
 - alternar_estado.php
 - eliminar_utilizador.php
 - processar_edicao_utilizador.php
 - processar_utilizador.php
 - utilizadores.php

public
index.php
login.php

/public/ (Área Pública e Autenticação)

Diretório de acesso livre. Acomoda o portal institucional (index.php), cujos conteúdos são extraídos dinamicamente da base de dados através de um CMS, e a interface de autenticação (login.php). Estes ficheiros formam a barreira de entrada da aplicação, validando credenciais antes de conceder acesso aos recursos internos.

/private/ (Área Reservada e Módulos)

O núcleo operacional do sistema, blindado por validações de sessão em PHP. Apenas utilizadores autenticados conseguem interagir com esta diretoria. Devido à dimensão do projeto, o *backend* foi modularizado em subdiretórios específicos para cada contexto hospitalar, destacando-se:

- /equipamentos/ e /localizacoes/ para o mapeamento do parque ativo.
- /armazem/ para o controlo de consumíveis, ruturas e encomendas.
- /manutencao/ para a gestão do ciclo de Ordens de Trabalho (preventivas e corretivas).
- /utilizadores/, /perfis/ e /historico/ para a administração de segurança e auditoria de ações.
- /relatorios/ para a exportação analítica de indicadores em formato CSV/Excel.

/assets/ (Recursos Estáticos e Identidade Visual)

Pasta dedicada aos ficheiros que suportam a interface gráfica, separada em recursos de terceiros (bootstrap.min.css, bibliotecas visuais) e código nativo. Em cumprimento com as diretrizes do projeto, a estilização e o comportamento personalizado do sistema foram isolados em ficheiros identificados com o número de estudante, nomeadamente **css/1241251.css** para regras de *design* adaptado (como a formatação de alertas de rutura) e **js/1241251.js** para os *scripts* lógicos globais de *frontend* (como a iniciação de paginação e validações assíncronas do cliente).

- **Raiz e Inclusões Globais:** Fora da estrutura de vistas, encontram-se os pilares transversais do projeto: o ficheiro db.php, que centraliza as conexões PDO e dezenas de funções utilitárias globais (validações e modelos de extração); e o layout.php, que utiliza o conceito de *template* para renderizar de forma DRY (*Don't Repeat Yourself*) o cabeçalho, as barras de navegação e as componentes modais partilhadas.

Front Office (Área Pública)

Conceito do Website Institucional

O portal público da aplicação atua como a montra digital da *Gira*, uma empresa simulada de *software* biomédico. Desenvolvido no ficheiro `index.php`, este módulo foi concebido para transmitir profissionalismo e confiança clínica, assumindo um forte foco na usabilidade e na acessibilidade.

A identidade visual tira partido do *framework* Bootstrap, garantindo um comportamento totalmente responsivo e limpo. A interface foi enriquecida com elementos dinâmicos suportados pela folha de estilos personalizada (`1241251.css`), onde foi implementada, por exemplo, uma animação CSS *keyframes* (carrossel contínuo) que apresenta as funcionalidades do sistema de forma fluida sem exigir interação manual.

Para dotar a simulação de um elevado grau de realismo, os conteúdos textuais foram aplicados ao contexto nacional. A plataforma apresenta informações de contacto institucionais, localizando a sede física no Porto (Rua Dr. António Bernardino de Almeida, 431) e disponibilizando contactos telefónicos (+351 228 340 500) e de correio eletrónico (`geral@gira.pt`) credíveis. Este portal serve ainda de ponte para a área restrita, encaminhando os utilizadores para a página de autenticação segura.

Mecanismo de Atualização Dinâmica

Um dos grandes elementos de valorização arquitetónica deste projeto foi a transição de um portal estático em HTML para uma plataforma gerida dinamicamente. Os textos, títulos, subtítulos e contactos visíveis na página inicial são extraídos em tempo real do motor de base de dados.

A nível de persistência, esta mecânica foi suportada pela tabela `conteudos_site`, estruturada num modelo ágil de chave-valor (com os atributos `chave` e `texto`). O ficheiro público `index.php` executa uma consulta a esta tabela durante o seu carregamento, mapeando os resultados num *array* associativo (`$conteudos`) para renderização imediata, possuindo inclusivamente blocos *try-catch* que garantem que o site não quebra visualmente caso exista uma falha na ligação à base de dados.

A gestão destes dados é efetuada no módulo privado `Backoffice Público` (`backoffice_publico.php`). Através de uma interface segmentada por separadores ("Secção Inicial", "Sobre Nós" e "Contactos"), o administrador do sistema pode editar os blocos de texto e submeter as alterações. O *script* de processamento (`processar_conteudos.php`) atua iterativamente sobre os dados recebidos via POST, aplicando as atualizações à base de dados. Para garantir a integridade total do portal, este processo foi blindado com Transações SQL (`$pdo->beginTransaction()` e `$pdo->commit()`). Esta lógica transacional assegura a atomicidade da operação: ou todos os campos (título, morada, e-mail) são atualizados em simultâneo com sucesso, ou, em caso de erro, a operação sofre uma reversão (`$pdo->rollBack()`), impossibilitando que a página pública apresente informações inconsistentes ou incompletas aos utilizadores.

Modelação e Engenharia da Base de Dados (MySQL)

Modelo Relacional e Esquema DBML

Para suportar o elevado volume de dados inerente à gestão de um parque tecnológico hospitalar, a base de dados relacional foi desenhada de raiz com foco na integridade e minimização de redundâncias. O modelo concetual foi convertido no esquema lógico abaixo (representado em linguagem DBML), permitindo a geração automática do Diagrama Entidade-Associação com a notação *Crow's Foot* (Pé de Galinha) (Figura 1), que dita a cardinalidade das chaves estrangeiras (relações 1:N e N:M).

```
// =====
// SISTEMA DE INFORMAÇÃO GIRA - DBML
// =====

Table perfis_acesso {
  id int [pk, increment]
  nome_perfil varchar(50) [not null]
  descricao varchar(255)
  nivel_acesso int [default: 1, not null]
  apagado_em datetime
}

Table utilizadores {
  id int [pk, increment]
  nome varchar(100) [not null]
  cedula varchar(50) [unique]
  email varchar(100) [unique, not null]
  password_hash varchar(255) [not null]
  perfil_id int [not null]
  estado varchar(20) [default: 'Ativo']
  data_criacao datetime [default: `CURRENT_TIMESTAMP`]
  ultimo_login datetime
  apagado_em datetime
}

Table fornecedores {
  id int [pk, increment]
  nome_empresa varchar(100) [not null]
  nif varchar(20) [unique, not null]
  especialidade varchar(100) [not null]
  estado varchar(20) [default: 'Ativo']
  email_suporte varchar(100)
  telefone_suporte varchar(20)
  apagado_em datetime
}

Table localizacoes {
  id int [pk, increment]
  cod_sala varchar(50) [unique, not null]
```

```

nome varchar(100) [not null]
detalhe varchar(100)
piso varchar(50)
bloco varchar(100)
apagado_em datetime
}

Table equipamentos {
  id int [pk, increment]
  codigo_ativo varchar(50) [unique, not null]
  nome varchar(100) [not null]
  categoria varchar(100)
  marca varchar(100)
  modelo varchar(100) [not null]
  num_serie varchar(100) [not null]
  mac_address varchar(50)
  classe_risco varchar(50)
  estado varchar(20) [default: 'Operacional']
  localizacao_id int
  fabricante_id int
  fornecedor_id int
  data_aquisicao date
  ano_fabrico int
  tipo_entrada varchar(20) [default: 'Compra']
  custo_aquisicao decimal(10,2)
  fim_garantia date
  proxima_revisao date
  apagado_em datetime
  observacoes text
}

Table documentos Equipamento {
  id int [pk, increment]
  equipamento_id int [not null]
  nome_documento varchar(255) [not null]
  tipo_documento varchar(100) [not null]
  caminho_ficheiro varchar(255) [not null]
  data_upload datetime [default: `CURRENT_TIMESTAMP`]
  apagado_em datetime
}

Table artigos_armazem {
  id int [pk, increment]
  referencia varchar(50) [unique, not null]
  nome varchar(100) [not null]
  categoria varchar(50) [not null]
  quantidade_atual int [not null, default: 0]
  quantidade_minima int [not null, default: 5]
  fornecedor_id int

```

```

    quantidade_em_transito int [not null, default: 0]
    apagado_em datetime
}

Table equipamento_artigo_armazem {
    equipamento_id int [pk]
    artigo_id int [pk]
}

Table encomendas_armazem {
    id int [pk, increment]
    artigo_id int [not null]
    quantidade int [not null]
    data_prevista date [not null]
    notas text
    estado varchar(50) [default: 'Pendente']
    data_pedido datetime [default: `CURRENT_TIMESTAMP`]
    apagado_em datetime
}

Table ordens_trabalho {
    id int [pk, increment]
    numero_ot varchar(50) [unique, not null]
    equipamento_id int [not null]
    tipo_manutencao varchar(100) [not null]
    prioridade varchar(50) [not null]
    descricao_averia text [not null]
    data_abertura datetime [default: `CURRENT_TIMESTAMP`]
    estado varchar(20) [default: 'Pendente']
    relatorio_tecnico text
    tempo_gasto decimal(5,1)
    data_fecho datetime
    apagado_em datetime
}

Table logs_auditoria {
    id int [pk, increment]
    data_hora datetime [default: `CURRENT_TIMESTAMP`]
    utilizador_id int
    acao varchar(255) [not null]
    modulo varchar(100) [not null]
    ip_origem varchar(45)
    tabela_afetada varchar(50)
    registo_id int
}

Table conteudos_site {
    chave varchar(50) [pk]
    texto text [not null]
}
    
```

```
}

// =====
// RELAÇÕES CROW'S FOOT (Foreign Keys)
// =====

Ref: utilizadores.perfil_id > perfis_acesso.id
Ref: equipamentos.localizacao_id > localizacoes.id
Ref: equipamentos.fornecedor_id > fornecedores.id
Ref: equipamentos.fabricante_id > fornecedores.id
Ref: documentos_equipamento.equipamento_id > equipamentos.id
Ref: artigos_armazem.fornecedor_id > fornecedores.id
Ref: equipamento_artigo_armazem.equipamento_id > equipamentos.id
Ref: equipamento_artigo_armazem.artigo_id > artigos_armazem.id
Ref: encomendas_armazem.artigo_id > artigos_armazem.id
Ref: ordens_trabalho.equipamento_id > equipamentos.id
Ref: logs_auditoria.utilizador_id > utilizadores.id
```

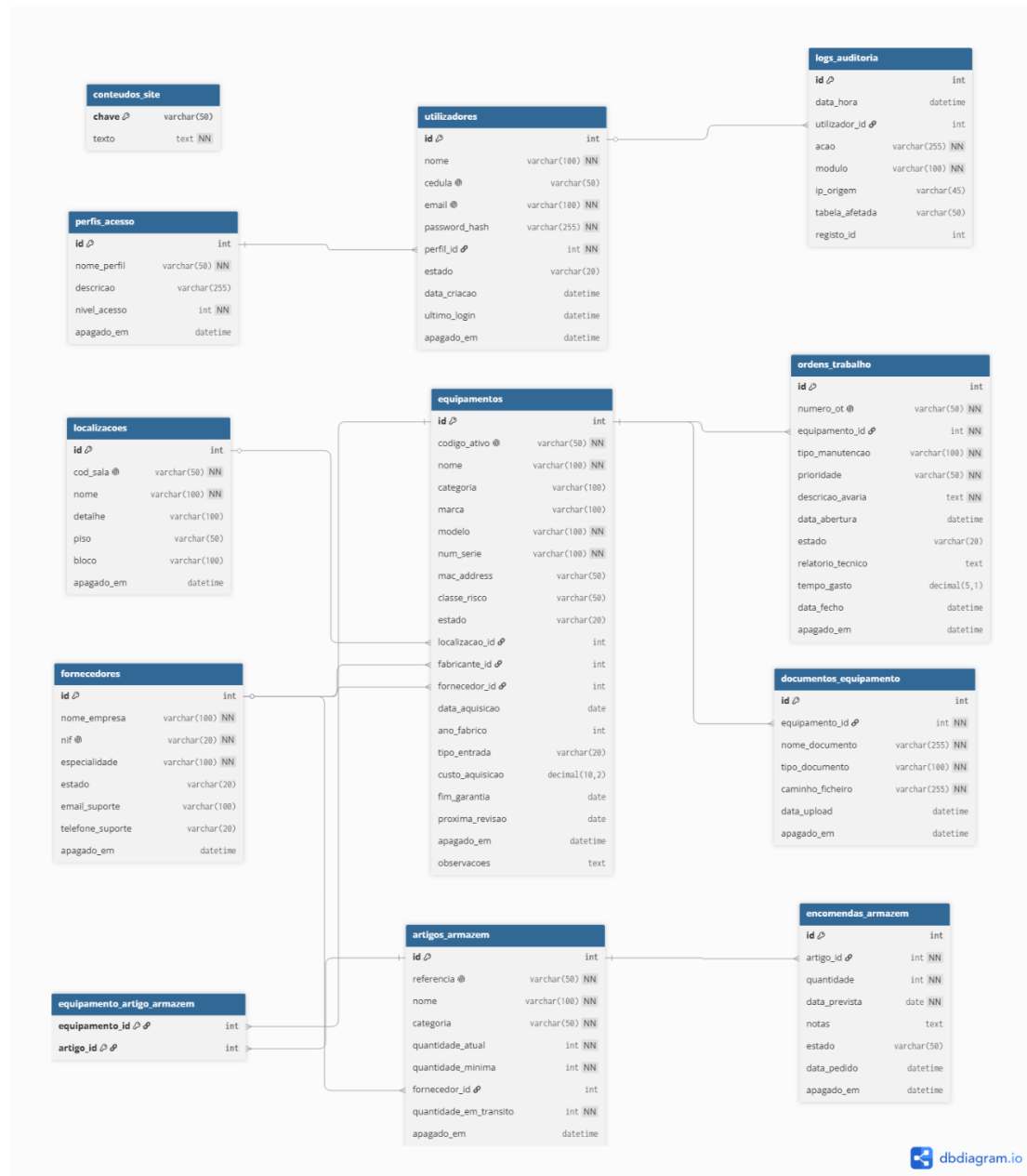


Figura 1 - Diagrama Entidade-Associação com a notação Crow's Foot (Pé de Galinha) da Gira.

Dicionário de Dados e Regras de Normalização

A conceção das tabelas baseou-se num rigoroso dicionário de dados que mapeou os tipos apropriados do MySQL para cada domínio: VARCHAR para identificadores alfa-numéricos (como NIFs ou Códigos de Ativos), DECIMAL(10,2) para valores monetários de aquisição, e tipos temporais DATE e DATETIME para controlo cronológico rigoroso (garantias, revisões e logs). Todas as tabelas possuem uma chave primária (id) numérica com auto-incremento (PK).

Para garantir a eficiência, o modelo foi sujeito aos princípios de Normalização de Dados:

- **1ª Forma Normal (1FN):** Eliminou-se a possibilidade de atributos multivalor (grupos repetidos). O cenário mais crítico consistia nos consumíveis/peças, uma vez que um equipamento pode usar várias peças e uma peça serve em vários equipamentos. Esta relação

N:M foi resolvida criando a tabela associativa equipamento_artigo_armazem, garantindo a atomicidade.

- **2ª e 3ª Formas Normais (2FN e 3FN):** As dependências transitivas e parciais foram removidas isolando as entidades em tabelas próprias. Por exemplo, os detalhes morfológicos de uma sala hospitalar (bloco, piso, nome) foram isolados na tabela localizacoes. Na tabela equipamentos, apenas figura a chave estrangeira (localizacao_id), prevenindo a duplicação textual sempre que se registem múltiplas máquinas na mesma sala. A mesma lógica foi aplicada aos fornecedores e perfis_acesso.

Restrições de Integridade Aplicadas no Modelo

Para garantir a consistência, segurança e fiabilidade dos dados do sistema Gira, o modelo relacional implementado recorre a um conjunto rigoroso de restrições de integridade. Estas regras previnem a duplicação de dados críticos e asseguram a integridade referencial entre as entidades em cenários de eliminação.

Restrições de Entidade (UNIQUE)

Para assegurar a identificação unívoca de registos fundamentais, foram aplicadas restrições do tipo UNIQUE nos seguintes atributos:

- **Tabela equipamentos:** O atributo codigo_ativo é único, garantindo que não existem dois ativos com a mesma chapa de inventário no parque tecnológico.
- **Tabela utilizadores:** Os atributos email e cedula são únicos, prevenindo o registo duplicado de profissionais de saúde.
- **Tabela fornecedores:** O atributo nif possui restrição de unicidade para evitar o registo múltiplo da mesma entidade comercial.
- **Tabelas localizacoes, artigos_armazem e ordens_trabalho:** Os atributos cod_sala, referencia e numero_ot também são únicos por imperativos de gestão física e logística.

Continuar em 2. Restrições de Integridade Referencial (Comportamento ON DELETE)

Restrições de Integridade Referencial (Comportamento ON DELETE)

O sistema gere as dependências entre tabelas primárias e secundárias utilizando comportamentos específicos aquando da eliminação de registos, evitando o surgimento de registos órfãos (violação de integridade referencial).

Eliminação em Cascata (ON DELETE CASCADE): Esta restrição foi aplicada em entidades de dependência existencial absoluta.

- Na tabela documentos_equipamento, a chave estrangeira equipamento_id possui eliminação em cascata. Se um equipamento for removido do sistema, todos os documentos técnicos associados a esse ID são automaticamente apagados.
- O mesmo comportamento aplica-se à tabela ordens_trabalho e à tabela associativa equipamento_artigo_armazem. A eliminação do equipamento mestre reflete-se na eliminação das suas intervenções e ligações de peças compatíveis.

Anulação Referencial (ON DELETE SET NULL): Aplicada em relações onde a entidade secundária deve persistir mesmo após a remoção da entidade primária.

- Na tabela equipamentos, a chave estrangeira fabricante_id assume este comportamento. Se um fornecedor/fabricante for eliminado da base de dados, os equipamentos hospitalares por ele produzidos mantêm-se intactos no inventário, passando o respetivo identificador de fabricante a assumir o valor nulo (NULL).
- Na tabela logs_auditoria, a eliminação de um utilizador coloca o utilizador_id a nulo, preservando o histórico de auditoria (registo da ação) por questões legais e de rastreabilidade.

Restrições Não Representáveis no Modelo Relacional

Embora o modelo relacional e o respetivo diagrama (notação *Crow's Foot*) sejam eficientes a ilustrar a cardinalidade e as dependências estruturais através de chaves estrangeiras, existem regras de negócio e restrições de domínio específicas que não podem ser expressas graficamente no esquema conceptual.

No sistema Gira, estas restrições mais complexas foram garantidas através da combinação de restrições de verificação (CHECK constraints, tipos de dados limitados como ENUM) no motor de base de dados e de funções de validação rigorosas na Camada de Aplicação (Backend em PHP). As principais restrições desta natureza aplicadas no projeto incluem:

Restrições de Formato e Padrão (Expressões Regulares)

O modelo relacional define atributos como o NIF da entidade fornecedora, o Endereço MAC do equipamento ou o Contacto Telefónico como cadeias de caracteres (VARCHAR), mas não tem capacidade gráfica para limitar o seu formato. Esta integridade é assegurada via código: o NIF e o Telefone são estritamente validados para conterem exatamente 9 dígitos numéricos, e o endereço MAC é forçado a respeitar a máscara de rede padrão (XX:XX:XX:XX:XX:XX).

Restrições Cronológicas e Temporais

O diagrama não representa a dependência temporal e lógica entre dois atributos do tipo DATE. No sistema, aplicou-se a restrição de que a "Data de Fim de Garantia" não pode ser cronologicamente anterior à "Data de Aquisição" do equipamento, assim como as datas previstas para a "Próxima Revisão" preventiva exigem referência no futuro.

Restrições de Domínio de Valores Lógicos

O sistema impede ações que fujam à lógica física de um hospital, nomeadamente:

- **Quantidades de Stock:** A quantidade_atual de um artigo de armazém não pode assumir valores negativos em resultado do abatimento de consumíveis.
- **Estados de Manutenção:** Não é possível emitir uma Ordem de Trabalho para um equipamento que já se encontre arquivado/abatido do inventário ativo (onde apagado_em IS NOT NULL), nem suspender um equipamento que já tenha o estado 'Inoperacional'.

Estas regras de negócio garantem a fiabilidade da informação muito além da estrutura base das tabelas, blindando a base de dados contra inserções incoerentes.

Consultas SQL Aplicadas

Em cumprimento dos requisitos de desenvolvimento, o sistema *Gira* tira partido das potencialidades da linguagem SQL para otimizar a extração de dados e garantir a integridade relacional. Apresentam-se de seguida duas consultas aplicadas na plataforma.

Consulta com Junção de Tabelas (JOIN)

A consulta abaixo foi implementada no ficheiro `detalhes Equipamento.php` para renderizar os cartões de *stock* de peças e consumíveis na ficha técnica da máquina.

Código SQL Aplicado:

```
$sql_cons = "SELECT a.* FROM artigos_armazem a
              INNER JOIN equipamento_artigo_armazem ea ON a.id =
ea.artigo_id
              WHERE ea.equipamento_id = :id_eq";
```

Esta consulta utiliza um INNER JOIN para extrair a lista de todos os artigos e peças do catálogo do armazém (`artigos_armazem`) que são compatíveis com o equipamento que o utilizador está a consultar. Como a relação entre os equipamentos médicos e as peças de substituição é de Muitos-para-Muitos (N:M) — uma vez que uma máquina usa várias peças e uma peça pode servir em várias máquinas — esta junção atua sobre a tabela associativa (`equipamento_artigo_armazem`), resolvendo a quebra da 1ª Forma Normal. A cláusula WHERE filtra dinamicamente os resultados utilizando o parâmetro: `id_eq` injetado de forma segura via PDO (*PHP Data Objects*).

Consulta com Subconsulta (Subquery)

As subconsultas foram aplicadas na plataforma para filtrar dados com base em condições de exclusão ou agregação em tabelas secundárias. A *query* abaixo foi implementada no painel analítico (`dashboard.php`) para identificar fragilidades documentais no inventário, respondendo diretamente a um dos indicadores de síntese exigidos.

Código SQL Aplicado:

```
$stmt_doc = $pdo->query
("SELECT COUNT(*)
FROM equipamentos
WHERE id NOT IN (
    SELECT DISTINCT equipamento_id
    FROM documentos_equipamento)");
```

Esta consulta utiliza uma subconsulta com o operador de exclusão NOT IN para calcular o número exato de equipamentos que não possuem qualquer manual, certificado ou ficheiro técnico

associado. O motor de base de dados resolve primeiro a pesquisa interna (isolando os IDs únicos de todos os equipamentos que possuem registos na tabela documentos_equipamento) e, de seguida, a consulta externa contabiliza (COUNT) todos os ativos da tabela mestre de equipamentos cujo ID não consta nesse vetor. Esta abordagem por subconsulta garante uma performance muito superior à de um *LEFT JOIN* com verificação de nulos para a geração de métricas estatísticas simples no *Dashboard*.

Back Office (Área Privada e Módulos Funcionais)

Autenticação e Segurança

A confidencialidade e a integridade dos dados são imperativos absolutos num sistema de Engenharia Clínica. Por essa razão, a plataforma Gira implementa um mecanismo rigoroso de autenticação e gestão de sessões nativas do PHP para blindar o acesso ao diretório `/private/`.

Mecanismo de Login e *Hashing* de Palavras-passe

A porta de entrada do sistema localiza-se na página `login.php`, pertencente à área pública. Quando um utilizador submete as suas credenciais, o *backend* interceta o pedido via método POST e cruza o endereço de e-mail com a tabela de utilizadores. Por razões de segurança, a base de dados não armazena as palavras-passe em texto limpo (*plaintext*), utilizando o algoritmo de encriptação irreversível suportado pela função nativa `password_hash()`. A autenticação é validada através da função `password_verify()`, que compara a credencial introduzida com o *hash* encriptado guardado na base de dados. Se a validação for bem-sucedida, o sistema regista a hora do acesso no campo `ultimo_login` e inicializa as variáveis globais de sessão (`$_SESSION`), armazenando de forma segura o ID do utilizador, o seu nome, e-mail e o seu nível de acesso.

Gestão de Sessões e Proteção de Diretórios

Para garantir que as páginas internas (CRUDs, relatórios e *dashboards*) não podem ser acedidas via manipulação direta de URLs, foi implementada uma barreira arquitetónica no ficheiro `layout.php` (que atua como cabeçalho mestre de toda a área reservada). Logo na primeira linha de execução deste ficheiro, a diretiva `session_start()` é invocada. O sistema verifica se a variável `$_SESSION['user_id']` está instanciada; caso não esteja, o utilizador é considerado um intruso e é acionado um redirecionamento forçado (`header("Location: ...")`) que o expulsa imediatamente para a página de login. Adicionalmente, as variáveis de sessão como `$_SESSION['nivel_acesso']` são utilizadas em vários módulos (como a eliminação de perfis ou criação de contas) para assegurar que apenas Administradores (nível 3) conseguem executar ações destrutivas ou de configuração global.

Processo de Encerramento (Logout)

O término seguro da utilização do sistema é gerido pelo *script* `logout.php`. Este processo atua a três níveis para garantir a erradicação da sessão:

1. Limpa todas as variáveis armazenadas no *array* global `$_SESSION`.
2. Força a invalidação do *cookie* de sessão residente no *browser* do utilizador (ajustando o seu tempo de vida para o passado através de `setcookie()`).
3. Destrói fisicamente a sessão no servidor através da invocação de `session_destroy()`, redirecionando finalmente o utilizador para a área pública.

Dashboard de Gestão (Painel Analítico)

O *Dashboard* atua como o painel de controlo central da aplicação Gira, fornecendo aos gestores e engenheiros clínicos uma visão global, imediata e em tempo real sobre o estado do parque tecnológico hospitalar.

Conceção Técnica e Lógica de Negócio

A nível de *backend*, o painel não recorre a dados estáticos. No momento em que a página `dashboard.php` é carregada, o controlador PHP executa um conjunto de consultas SQL de agregação (recorrendo à função `COUNT()`) para extrair métricas exatas da base de dados. Estes indicadores-chave de desempenho (KPIs) incluem:

- **Dimensão do Parque e Taxa de Disponibilidade:** O sistema calcula o total absoluto de equipamentos e cruza essa informação com os ativos em estado 'Operacional', gerando dinamicamente a percentagem da taxa de disponibilidade (`$taxa_disp`).
- **Carga de Manutenção:** Uma consulta dedicada à tabela `ordens_trabalho` contabiliza todas as intervenções cujo estado difere de 'Concluída', alertando para o volume de reparações pendentes.
- **Alertas de Garantia:** O sistema avalia o campo `fim_garantia` dos equipamentos, filtrando e contabilizando os ativos cujas coberturas legais expiram num prazo inferior a 30 dias.

Conceção Visual e Interface Gráfica

Para a apresentação destes dados, optou-se por não integrar bibliotecas externas de renderização gráfica (como o *Chart.js* ou *ApexCharts*). Esta decisão arquitetónica visou maximizar a velocidade de carregamento da página e manter a leveza do DOM (*Document Object Model*).

Em alternativa, a interface gráfica foi construída utilizando os componentes nativos do *framework* Bootstrap 5 (nomeadamente o sistema de grelhas e os elementos *Card*). Os indicadores são apresentados através de blocos numéricos de alto contraste, complementados por iconografia vetorial da biblioteca *FontAwesome* (para rápida perceção visual do contexto da métrica) e componentes interativos (como botões de ação rápida para "Criar Utilizador" ou "Adicionar Perfil"), garantindo um *design* limpo, utilitário e alinhado com as necessidades de um ambiente clínico de ritmo acelerado.

Módulos de Gestão Operacional (CRUDs)

Os módulos operacionais constituem o núcleo transacional da plataforma Gira. A arquitetura de interface transversal a todos estes módulos baseia-se na renderização de listagens dinâmicas suportadas pela biblioteca *DataTables* (garantindo paginação e pesquisa no lado do cliente) e na utilização de *Modais* do Bootstrap para as ações de Inserção e Edição. Esta abordagem evita o carregamento desnecessário de novas páginas, mantendo o utilizador no mesmo contexto visual.

A nível de *backend*, o processamento dos formulários é assíncrono à visualização, recorrendo a *scripts* dedicados (ex: `processar_equipamento.php`) que validam os dados, executam a transação via PDO e utilizam redirecionamentos inteligentes (`HTTP_REFERER`) para devolver o utilizador à listagem com mensagens de sucesso ou de erro encapsuladas em variáveis de sessão.

Gestão de Equipamentos Médicos

Este é o módulo de maior complexidade relacional do sistema. A ficha técnica de um equipamento engloba não só identificadores únicos (Código de Ativo, Número de Série, Endereço MAC) e dados financeiros (Custo de Aquisição), mas também a sua interligação com outras entidades.

- **Inserção e Edição:** Durante o registo, o formulário recolhe chaves estrangeiras para definir a localização exata da máquina e os seus parceiros comerciais (distinguindo claramente quem é o Fabricante e quem é o Fornecedor de Assistência Técnica). Adicionalmente, permite a seleção múltipla de consumíveis e peças (provenientes do módulo de Armazém), gravando estas ligações numa tabela associativa.
- **Remoção e Suspensão:** Em vez da destruição física de dados (DELETE), o sistema aplica um **Abate Lógico**. Quando um equipamento é eliminado, o campo apagado_em regista a data de purga, ocultando a máquina das listagens ativas, mas preservando o seu histórico para auditoria. Foi também implementada uma função de "Suspensão", que altera o estado do ativo para 'Inoperacional' sem o remover do parque.

Gestão de Localizações (Mapeamento Físico)

O módulo de localizações estrutura o mapeamento físico do hospital, permitindo categorizar as áreas em cascata (Bloco/Edifício > Piso > Serviço > Sala).

- **Estrutura de Dados:** A criação de um novo espaço exige um cod_sala único, prevenindo ambiguidades logísticas.
- **Defesa de Eliminação:** Foi programada uma regra de negócio crítica no *script* eliminar_localizacao.php. Antes de permitir o abate de uma sala, o sistema conta (COUNT) quantos equipamentos operacionais estão associados a esse ID. Se a sala estiver ocupada, o abate é bloqueado e o utilizador é notificado, prevenindo o surgimento de equipamentos "flutuantes" ou órfãos no hospital.

Gestão de Fornecedores e Contratos

Esta vertente gere o ecossistema de parceiros externos do departamento de Engenharia Clínica.

- **Registo e Distinção Comercial:** O CRUD permite o registo de empresas com validação estrita do NIF (impedido duplicações via constrangimento UNIQUE). Na perspetiva de arquitetura, a tabela de fornecedores foi desenhada para atuar de forma versátil: a mesma entidade pode ser associada a um equipamento como "Fabricante" (quem construiu a máquina) ou como "Assistência Técnica / Distribuidor Comercial" (quem a vendeu ou repara), garantindo uma rastreabilidade comercial completa.
- **Segurança na Remoção:** À semelhança das localizações, o abate de um fornecedor está protegido. O sistema aborta qualquer tentativa de eliminação se a empresa tiver contratos de assistência ou fabrico ativos associados a equipamentos no inventário.

Gestão Documental e Uploads

A digitalização de manuais de utilizador, esquemas elétricos e certificados de calibração é um requisito crítico na Engenharia Clínica. Para o módulo de Gestão Documental do sistema Gira, afastou-se a abordagem simplificada de inserção de hiperligações externas, optando-se pelo desenvolvimento de um mecanismo real de *upload* de ficheiros (maioritariamente PDFs) para o servidor.

Metodologia de Armazenamento (Sistema de Ficheiros vs. BLOB)

A nível arquitetónico, tomou-se a decisão de não armazenar os ficheiros fisicamente dentro da base de dados (utilizando o tipo de dados BLOB - *Binary Large Object*). Essa prática tornaria a base de dados excessivamente pesada e lenta. Em vez disso, o *script* de processamento (processar_documento.php) utiliza a variável superglobal \$_FILES do PHP para intercepar o documento submetido, validar o seu peso face aos limites do servidor (CONTENT_LENGTH), e movê-lo fisicamente para um diretório seguro dentro da pasta /private/uploads/. Na base de dados (tabela documentos_equipamento), é guardado apenas o identificador textual que aponta para esse ficheiro no disco (o atributo caminho_ficheiro), garantindo uma leitura leve e rápida.

Associação Relacional ao Parque Tecnológico

A associação de um PDF ao seu respetivo equipamento é garantida de forma estrita através da chave estrangeira equipamento_id. Quando o *upload* é efetuado a partir da ficha técnica de uma máquina, o formulário captura silenciosamente o ID desse equipamento via campo oculto (<input type="hidden">). O motor de base de dados assegura a integridade desta ligação; graças ao comportamento ON DELETE CASCADE configurado no modelo relacional, a associação é indestrutível e à prova de registos órfãos. Adicionalmente, o sistema permite a inserção de "Documentos Gerais" (onde o equipamento_id é nulo), servindo como repositório para normativas hospitalares transversais.

Gestão de Espaço e Remoção Física (unlink)

Ao contrário da maioria dos módulos do sistema que utilizam exclusivamente o conceito de "Abate Lógico" (apagado_em = NOW()), a eliminação de um documento técnico desencadeia uma ação física no servidor. O *script* eliminar_documento.php executa a função nativa unlink() do PHP, que destrói permanentemente o ficheiro alojado no disco do servidor antes de atualizar o estado na base de dados. Esta lógica de engenharia é vital para prevenir o esgotamento progressivo do armazenamento do servidor com PDFs obsoletos ao longo dos anos de utilização hospitalar.

Pesquisa Avançada e Filtragem Combinada

A localização ágil de um equipamento médico, especialmente em cenários de emergência clínica ou auditoria, é um requisito não funcional imperativo. Para responder a esta necessidade sem comprometer a performance do servidor com sucessivos pedidos à base de dados, a plataforma Gira adotou uma arquitetura de pesquisa e filtragem com processamento do lado do cliente (*Client-Side Processing*).

Arquitetura do Motor de Busca (DataTables)

Em vez de desenvolver múltiplos formulários com consultas SQL dependentes de cláusulas LIKE e AND para cada variação de filtro, a abordagem tecnológica centrou-se na biblioteca JavaScript **DataTables**. O fluxo de dados funciona da seguinte forma: o controlador PHP (`equipamentos.php`) executa uma única consulta de extração global (SELECT) no momento do carregamento da página e constrói a tabela HTML estruturada no DOM (*Document Object Model*) através das funções centrais alojadas no `layout.php`. De seguida, o ficheiro de *scripts* nativo (`1241251.js`) interceta a classe CSS identificadora da tabela (`.gira-datatable`) e inicializa o motor dinâmico sobre a mesma.

Lógica de Filtragem Combinada e Cruzamento de Dados

A caixa de pesquisa global implementada atua como um motor de filtragem inteligente multiplataforma (conhecido como *Smart Search*). Ao contrário de uma pesquisa exata, o algoritmo fragmenta os termos introduzidos pelo utilizador e procura a sua correspondência em qualquer coluna da tabela de forma simultânea.

Isto permite aos gestores hospitalares efetuar cruzamentos complexos de informação sem recorrer a painéis de filtros extensos. Por exemplo, se o engenheiro clínico introduzir a sequência de texto "Dräger Suporte Vida Inoperacional", o motor avalia instantaneamente o DOM, ocultando todas as linhas (*rows*) que não satisfaçam esta interseção. O resultado é uma tabela filtrada em tempo real (latência zero) que cruza atributos de entidades diferentes: a Marca (Dräger), a Classe de Risco (Suporte de Vida) e o Estado do ativo (Inoperacional).

Esta decisão garante três mais-valias fundamentais para o sistema:

- **Performance do Servidor:** Elimina a necessidade de *round-trips* (pedidos contínuos) entre o *browser* e o servidor MySQL sempre que o utilizador altera um critério de pesquisa.
- **Paginação Dinâmica:** À medida que a pesquisa é refinada, a biblioteca recalcula automaticamente a paginação e o total de registos visíveis no rodapé da tabela.
- **Escalabilidade Visual:** A ordenação alfabética, cronológica e numérica das colunas é garantida de forma nativa e bidirecional pelos algoritmos da biblioteca, poupando centenas de linhas de código PHP auxiliar.

Funcionalidades Opcionais e de Valorização

Com o intuito de aproximar a plataforma Gira da realidade de um sistema de Gestão de Manutenção Assistida por Computador (CMMS) de nível profissional, o desenvolvimento deste projeto ultrapassou os requisitos mínimos exigidos. Foram concebidas e implementadas funcionalidades avançadas focadas na segurança, rastreabilidade e experiência do utilizador.

Exportação Analítica (Relatórios em Excel/CSV)

Para dotar o sistema de verdadeira utilidade de gestão, foi criado um módulo capaz de exportar os dados do inventário e das intervenções técnicas para formatos estruturados. A nível de *backend* (exportar_relatorio.php), o PHP atua diretamente sobre o fluxo de saída (*output stream*), alterando os cabeçalhos HTTP para forçar o *download* (Content-Disposition: attachment). Uma das valorizações técnicas desta implementação foi a injeção do *Byte Order Mark* (BOM) do UTF-8 na primeira linha do ficheiro (chr(0xEF) . chr(0xBB) . chr(0xBF)), garantindo que o Microsoft Excel em sistemas Windows reconhece corretamente os caracteres especiais e acentuações da língua portuguesa gerados pelo fputcsv.

Motor de Auditoria (Logs) e Restauro de Dados (*Soft Deletes*)

Em sistemas clínicos, a destruição irrecuperável de dados é um risco grave. Por essa razão, a instrução SQL DELETE foi abolida de quase todo o projeto (exceto no descarte físico de ficheiros PDF). Adotou-se o conceito de **Abate Lógico** (*Soft Delete*): quando um registo é eliminado, o sistema preenche a coluna apagado_em com a data atual, ocultando-o das vistas públicas, mas mantendo a integridade referencial. Paralelamente, foi desenvolvida uma tabela cega de logs_auditoria. Sempre que ocorre uma transação (inserção, edição ou abate), o sistema regista quem a fez, a que horas e em que módulo. Foi ainda programado um *script* de restauro (restaurar_registro.php) que permite aos administradores reverter abates acidentais, recolocando o campo apagado_em a nulo e reativando o equipamento ou utilizador instantaneamente.

Armazém Técnico e Alertas de Rutura

A gestão de equipamentos foi expandida para incluir a logística de consumíveis e peças de substituição, através do desenvolvimento de um módulo de Armazém completo. A arquitetura da base de dados passou a relacionar as peças com os equipamentos compatíveis (associação N:M). Para valorizar o *Dashboard*, o código interceta os artigos cuja quantidade_atual é inferior ao limiar da quantidade_minima, disparando automaticamente alertas visuais de rutura de *stock* para os engenheiros efetuarem as respetivas encomendas.

Calendário Dinâmico de Manutenções

Para facilitar a visualização da carga de trabalho preventivo, o módulo de manutenção foi enriquecido com a renderização de um calendário interativo. Através da conversão do vetor de resultados do PHP para um objeto JSON, os dados das "Próximas Revisões" são injetados diretamente na biblioteca JavaScript *FullCalendar*. O calendário apresenta visualmente os prazos de conformidade legal de cada máquina, permitindo uma navegação intuitiva por meses e semanas.

Segurança Transaccional (Propriedades ACID)

Em operações críticas de múltiplos passos, como o encerramento de uma Ordem de Trabalho (onde o sistema tem de atualizar o histórico da intervenção e, simultaneamente, mudar o estado do equipamento para "Operacional"), foi implementada a mecânica de Transações PDO. O processo é encapsulado entre `$pdo->beginTransaction()` e `$pdo->commit()`. Caso ocorra uma falha de servidor a meio da operação, o bloco catch executa um `$pdo->rollBack()`. Esta engenharia assegura a atomicidade e previne absolutamente a corrupção parcial de dados na base de dados relacional.

Interface Adaptativa (Modo Escuro / *Dark Mode*)

Tendo em conta as exigências ergonómicas dos profissionais de saúde (que frequentemente operam o *software* em ambientes de baixa luminosidade, como blocos operatórios ou postos de imagiologia), foi implementada uma versão de *Dark Mode*. A transição gráfica recorre à manipulação do DOM e a variáveis CSS personalizadas (ou atributos *data-theme* do Bootstrap), com o JavaScript a memorizar a preferência do utilizador através do LocalStorage do *browser*. Isto assegura que a experiência visual preferida persiste durante a navegação entre as várias páginas do Back Office sem causar fadiga ocular.

Controlo de Versões e Histórico de Desenvolvimento (Git)

Metodologia de Trabalho

A construção de um sistema com a complexidade do Gira exigiu uma gestão rigorosa do código-fonte para prevenir a perda de dados e permitir a reversão segura de alterações estruturais. Para este efeito, foi adotado o sistema de controlo de versões **Git**, inicializado localmente na diretoria do projeto (`git init`).

A metodologia de trabalho baseou-se em ciclos de desenvolvimento iterativos. Cada nova funcionalidade (como a criação de um CRUD ou a implementação do *Dashboard*) foi desenvolvida, testada e posteriormente encapsulada na base de código através de comandos de rastreamento (`git add .`) e submissão (`git commit`).

No que diz respeito à identificação de autoria, o ambiente Git foi configurado na estação de trabalho local com as credenciais do programador. Embora o projeto se insira num contexto académico, a diretiva `git config global user.email` foi parametrizada com o endereço de correio eletrónico pessoal (prática padrão em máquinas de desenvolvimento particular e em plataformas como o GitHub). A integridade e a autoria académica do trabalho foram, contudo, inequivocamente asseguradas através da configuração estrita da diretiva `git config global user.name` com a identificação "Simão Cacheira", garantindo que todas as assinaturas criptográficas dos *commits* refletem o autor do projeto.

Análise de Commits e Comprovação de Evolução

O histórico do repositório espelha o desenvolvimento atómico e progressivo do sistema, rejeitando a prática de submissões monolíticas no final do projeto. A árvore de *commits* evidencia uma separação clara de responsabilidades, cumprindo e ultrapassando largamente o requisito mínimo de 30 *commits* exigido pelo guião da unidade curricular.

A cronologia das submissões reflete as duas grandes frentes da aplicação:

- **Evolução do Frontend (Aprox. 15+ Commits):** Submissões focadas na construção da Interface de Utilizador (UI). O histórico demonstra a evolução desde a estrutura HTML em bruto, passando pela injeção do *framework* Bootstrap 5, o desenho do ficheiro `1241251.css`, até à implementação lógica de JavaScript no cliente (`1241251.js`), como as inicializações do *DataTables* e a gestão de modais.
- **Evolução do Backend (Aprox. 15+ Commits):** Submissões orientadas à lógica de negócio e persistência. Os registos comprovam a construção do `db.php`, a criação do `layout.php` para proteção de sessões e a implementação progressiva das instruções SQL (PDO) e verificações de segurança em cada *script* de processamento da pasta `/private/`.

Para efeitos de auditoria e validação por parte do corpo docente, o histórico integral de submissões foi exportado através do terminal (recorrendo ao comando `git log --oneline > commits.txt`). Este ficheiro de evidências, denominado **commits.txt**, encontra-se anexado e disponível na raiz do diretório principal do projeto submetido.

Conclusões e Trabalho Futuro

Considerações Finais

O desenvolvimento do sistema Gira representou um marco fundamental na consolidação de competências técnicas e analíticas, essenciais no panorama da Engenharia Biomédica moderna. A transição de conceitos teóricos de modelação de dados para a construção de uma plataforma *web* plenamente funcional demonstrou, na prática, como a tecnologia orientada aos sistemas de informação é capaz de resolver constrangimentos logísticos complexos em ambiente hospitalar.

A conceção desta plataforma permitiu materializar a ponte entre a integridade das bases de dados relacionais e a fluidez dos ambientes *web*. A integração rigorosa entre o motor MySQL — pautado por regras estritas de normalização, restrições de chaves estrangeiras e transações seguras — e o *backend* em PHP resultou numa solução de alta fiabilidade. Mais do que o desafio da escrita de código, este projeto exigiu uma constante adaptação e empatia para com o utilizador final, garantindo que a usabilidade da interface facilitaria o dia a dia de médicos, técnicos e engenheiros clínicos. O resultado final materializa-se num ecossistema robusto, que não só erradica a dependência de processos analógicos e folhas de cálculo vulneráveis, como estabelece um novo padrão para o controlo seguro e rastreável do parque tecnológico.

Limitações e Propostas de Melhoria

Apesar de a aplicação Gira ter cumprido e largamente ultrapassado os objetivos e requisitos iniciais, a sua arquitetura foi desenhada de forma modular, preparando o terreno para futuras expansões. De forma a elevar o projeto a uma ferramenta comercial de excelência, identificam-se as seguintes oportunidades de melhoria e trabalho futuro:

- **Evolução para um CMMS/GMAC Pleno:** Embora o atual módulo de manutenção suporte o registo e o encerramento de Ordens de Trabalho (O.T.s), a plataforma tem o potencial de evoluir para um autêntico sistema de Gestão de Manutenção Assistida por Computador. Este salto envolveria a automatização do plano preventivo, onde o próprio sistema gera as O.T.s e as aloca à agenda de técnicos específicos à medida que a data da "Próxima Revisão" se aproxima.
- **Sistema de Notificações Ativas:** O *Dashboard* atual exige que o utilizador inicie sessão para visualizar os alertas críticos. Uma melhoria substancial passaria pela integração de uma biblioteca de correio eletrónico (como o *PHPMailer*) para o disparo de *e-mails* automáticos à equipa de engenharia sempre que um equipamento de Suporte de Vida avaria, o *stock* de uma peça entra em rutura ou uma garantia está a menos de 30 dias de expirar.
- **Geração de Etiquetas e Leitura por QR Code:** Adição de uma rotina de geração de PDFs (recorrendo a bibliotecas como *DomPDF* ou *FPDF*) para a criação de etiquetas de inventário físicas. Ao dotar cada máquina de um QR Code gerado pelo sistema Gira, um técnico poderia, através de um *smartphone*, ler a etiqueta no corredor do hospital e aceder instantaneamente à ficha técnica completa e ao histórico clínico do ativo.
- **Integração na Rede Hospitalar (IoT):** Tirando partido do atributo "Endereço MAC" já existente no registo dos equipamentos, o sistema poderia, futuramente, executar *scripts* automatizados de *ping* na infraestrutura de rede para confirmar se as máquinas conectadas estão efetivamente *online* e a transmitir dados, atualizando a "Taxa de Disponibilidade" do *Dashboard* em tempo absoluto.

Referências

Para o desenvolvimento das soluções técnicas e da arquitetura do sistema *Gira*, foram consultadas as documentações oficiais das tecnologias implementadas e as boas práticas de desenvolvimento *web* recomendadas na unidade curricular.

- **Bootstrap Team.** (2026). *Bootstrap 5 Documentation*. Disponível em: <https://getbootstrap.com/>
- **DataTables.** (2026). *DataTables: Table plug-in for jQuery*. Disponível em: <https://datatables.net/>
- **FullCalendar.** (2026). *FullCalendar: Open source JavaScript Event Calendar*. Disponível em: <https://fullcalendar.io/>
- **MySQL AB / Oracle Corporation.** (2026). *MySQL 8.0 Reference Manual*. Disponível em: <https://dev.mysql.com/doc/>
- **PHP Group.** (2026). *PHP: Hypertext Preprocessor - Manual Oficial*. Disponível em: https://www.php.net/manual/pt_BR/
- **Unsplash.** (2026). *Imagens de alta qualidade para uso livre*. Disponível em: <https://unsplash.com/>